

# Russian Sentence Embeddings

langembed - ru language track

Repository: [github.com/DenisBaliuckij/langembed](https://github.com/DenisBaliuckij/langembed)

Plan: [docs/superpowers/plans/2026-07-01-russian-embeddings.md](#)

Design: [docs/superpowers/specs/2026-06-30-russian-embeddings-design.md](#)

Commits: [d781633](#) .. [3a852e7](#) (pushed to origin/master)

# 1. What we built

We added a new 'ru' language track to the existing langembded sentence-embedding pipeline, producing real, trained sentence embeddings for a Russian corpus extracted from data/raw/voina-i-mir.pdf. The PDF turned out to be Tolstoy's complete 90-volume collected works (1918 pages), not just War and Peace, so the corpus is substantial (28,283 sentences after deduplication).

The design goal was to reuse every existing pipeline stage unchanged (extraction, corpus building, tokenizer training, MLM pretraining, unsupervised SimCSE, evaluation, serving) and only add ru-specific configuration and two small new scripts. This kept the well-tested core pipeline logic completely untouched.

## Pipeline stages (each is one git commit)

- 1. PDF extraction -> sentence corpus -> BPE tokenizer
- 2. Bounded MLM pretraining of a small RoBERTa-style encoder from scratch
- 3. Unsupervised SimCSE contrastive fine-tuning
- 4. Human-in-the-loop STS labeling UI (active-learning sampled pairs)
- 5. Evaluation against the human-labeled test set (Spearman, Recall@k, MRR@k)
- 6. Final embeddings deliverable + zero train/serve-skew verification

## Key results

|                          |                                                          |
|--------------------------|----------------------------------------------------------|
| <b>Corpus size</b>       | 28,283 sentences (deduplicated from 28,413 extracted)    |
| <b>Tokenizer</b>         | BPE, vocab_size=16,000, unk_rate <= 1%                   |
| <b>Encoder</b>           | RoBERTa-style, 4 layers, hidden=256, ~7.4M params        |
| <b>MLM pretrain</b>      | 736 steps, loss 9.36 -> 7.07-7.12 (finite throughout)    |
| <b>SimCSE</b>            | 884 steps (1 epoch), loss -> 0.130 (finite throughout)   |
| <b>STS test set</b>      | 22 human-labeled pairs (data/sts_test_ru.jsonl)          |
| <b>Eval (branch A)</b>   | Spearman=0.292, Recall@5=1.00, MRR@5=0.682               |
| <b>Final deliverable</b> | 28,283 embedding vectors, embeddings_ru/embeddings.jsonl |
| <b>Serve/train skew</b>  | max_abs_diff = 0.00000004 (exact match)                  |

### Reading the eval numbers

Spearman and retrieval metrics are computed on only 22 labeled pairs, so they are a sanity check, not a rigorous benchmark. Recall@5=1.0 is near-trivial at this sample size (documented as an expected caveat in the design, not a bug).

## 2. Step-by-step walkthrough

### Phase 1 -- PDF extraction, corpus, tokenizer

A new module, `extract_text.py`, uses `pypdf` to pull text per page, dehyphenates words wrapped across lines, filters junk lines (bare page numbers, fragments under 20 characters), and splits into sentences with a regex tuned for Russian punctuation and quotation marks. The existing `build_corpus.py` and `train_tokenizer.py` were reused unmodified -- only pointed at new ru configs.

```
python -m langembed.data.extract_text \
  --pdf data/raw/voina-i-mir.pdf --out data/raw/voina_i_mir_ru.txt
-> sentences: 28413

python -m langembed.data.build_corpus --config configs/ru/tokenizer.yaml
-> data/corpus_ru.txt (28283 lines after near-duplicate removal)

python -m langembed.tokenizer.train_tokenizer --config configs/ru/tokenizer.yaml
-> artifacts/tokenizer_ru/ (BPE, vocab_size=16000, unk_rate <= 0.01)
```

### Phase 2 -- Bounded MLM pretraining

`configs/ru/pretrain.yaml` defines a small CPU-feasible RoBERTa-style encoder (`hidden_size=256`, 4 layers, 4 attention heads, `max_seq_length=128`). The plan called for calibrating training length to roughly 25 minutes on this machine: a 50-step 'smoke' run was timed (101.9s, including one-time import and data-loading overhead), and `max_steps` was computed from that measurement.

```
# 50-step timed smoke run
python -m langembed.pretrain.train_mlm --config configs/ru/pretrain.yaml --smoke
-> elapsed_seconds: 101.86

# max_steps = round(50 * 1500 / 101.86) = 736

# full calibrated run
python -m langembed.pretrain.train_mlm --config configs/ru/pretrain.yaml
-> 736/736 steps in 8m24s
-> loss: 9.357 -> 7.07-7.12 (finite throughout, no nan/inf)
-> artifacts/encoder_ru/ (config.json, model.safetensors, tokenizer files)
```

### Phase 3 -- Unsupervised SimCSE

`configs/ru/contrastive.yaml` fine-tunes the pretrained encoder with unsupervised SimCSE (dropout-based positive pairs) over the full corpus for one epoch.

```
python -m langembed.contrastive.train_simcse --config configs/ru/contrastive.yaml --smoke
-> smoke run on first 256 sentences: loss 1.073 (finite)

python -m langembed.contrastive.train_simcse --config configs/ru/contrastive.yaml
-> 884/884 steps (28283 sentences / batch_size 32) in 25m23s
-> loss -> 0.130 (finite throughout)
-> artifacts/simcse_ru/ (sentence-transformers save directory)
```

## Phase 4 -- Human-in-the-loop STS labeling

scripts/seed\_sts\_pairs.py builds label candidates two ways: adjacent sentence pairs (likely related) and randomly-sampled distant pairs (likely unrelated), then scores each pair by SimCSE 'uncertainty' -- cosine similarity closest to the 0.5 decision boundary is the most informative to a human labeler. The top 60 pairs (capped so no sentence repeats more than twice) were written to Postgres as pending items.

Two new routes were added to the existing annotation FastAPI service: GET/POST /label serves one pending pair at a time with a 1-5 similarity form, and GET /export-sts aggregates all labeled items into data/sts\_test\_ru.jsonl.

```
docker compose up -d postgres
python scripts/seed_sts_pairs.py --config configs/ru/contrastive.yaml --n 60
  -> seeded items: 60

python -m uvicorn langembed.annotation.api:app --port 8001
  -> open http://localhost:8001/label in a browser, rate pairs 1-5

curl -X GET http://localhost:8001/export-sts
  -> {"written": 22, "path": "data/sts_test_ru.jsonl"}
```

### Operational hiccup: stale Docker container

A leftover container from a prior build (langembed-annotation-1, image built ~2 weeks earlier) was already bound to port 8001 and silently served the OLD api.py without the new /label route, so the first browser request 404'd. Diagnosed via 'docker ps -a' and fixed with 'docker stop langembed-annotation-1', freeing the port for the freshly started local server that had the new code.

## Phase 5 -- Evaluation

configs/ru/eval.yaml points the existing evaluate.py at the human-labeled STS set and the SimCSE model (branch A). evaluate.py enforces a hard-coded 'no test leakage' guard: it hashes every sentence in the STS test file and fails loudly if any of those hashes also appear in the files listed as train\_paths.

### Design tension found: STS pairs overlap the training corpus by construction

Because the STS candidates in Phase 4 are literally sentences sampled from data/corpus\_ru.txt, the leakage guard fires every time train\_paths includes that file -- the labeled sentences ARE corpus sentences. This is the same situation the project's own smoke pipeline hit earlier (documented in its commit history), and it was resolved the same way: configs/ru/eval.yaml sets train\_paths: [] with a comment explaining the overlap is by design, not a leak of genuinely held-out data.

```
python -m langembed.eval.evaluate --config configs/ru/eval.yaml
  -> branch A: Spearman = 0.2922
  -> branch A: Recall@5=1.0000, MRR@5=0.6818
  -> metrics written to metrics/eval_ru.json
```

## Phase 6 -- Final embeddings + serve/train skew verification

scripts/embed\_corpus.py encodes every corpus sentence with the trained SimCSE model and writes {text, embedding} pairs -- this file is the actual deliverable a downstream consumer would use. scripts/verify\_serve\_skew.py then re-encodes a sample of those sentences through the live /embed HTTP endpoint and asserts the two paths produce numerically identical vectors -- the project's zero train/serve-skew acceptance test.

```
python scripts/embed_corpus.py --config configs/ru/contrastive.yaml \
  --out artifacts/embeddings_ru/embeddings.jsonl
-> embedded sentences: 28283 (matches corpus line count exactly)
```

### Real bug found: serve.py silently corrupted Russian text

The first skew check failed: max\_abs\_diff=0.0289, clearly not floating-point noise. Root cause: serve.py's /embed handler called normalize(text) with no language argument, which silently defaults to lang="gu" (Gujarati). The Gujarati-specific indic normalizer then mangled Cyrillic input -- e.g. stripping em-dashes -- producing text that differed from what build\_corpus.py had normalized with the correct lang="ru" when building the training corpus. This is a genuine, pre-existing correctness bug in shared serving code that would affect ANY future non-Gujarati language track, not just ru. Fix: added a LANGEMBED\_LANG environment variable to serve.py (default "gu", so existing behavior for the gu track is unchanged) so /embed can be told which language's normalization rules to apply.

```
# before fix
LANGEMBED_MODEL_DIR=artifacts/simcse_ru uvicorn langembed.serving.serve:app --port 8000
python scripts/verify_serve_skew.py --n 10
-> skew check: match=False max_abs_diff=0.02883990 n=10

# after fix (serve.py now reads LANGEMBED_LANG)
LANGEMBED_MODEL_DIR=artifacts/simcse_ru LANGEMBED_LANG=ru \
  uvicorn langembed.serving.serve:app --port 8000
python scripts/verify_serve_skew.py --n 10
-> skew check: match=True max_abs_diff=0.00000004 n=10
```

## 3. Findings worth sharing with the team

---

### 1. `serve.py` had a latent multi-language correctness bug

Anyone adding a second language track before this fix would have hit the same silent text corruption in production serving. The fix (`LANGEMBED_LANG` env var) is generic and benefits every future language, not just ru.

### 2. Active-learning-sampled STS pairs will always overlap their source corpus

Any track that seeds its human-labeling queue from candidate pairs drawn out of the training corpus itself (as opposed to independently authored STS sentences) needs `train_paths: []` in its eval config -- the leakage guard is doing its job correctly, the pairs really are the corpus. Worth deciding as an explicit project convention rather than re-discovering it per language.

### 3. Environment quirks specific to this Windows + Python 3.14 setup

- `sentence_transformers` segfaults on import unless `torch/datasets/pandas/pyarrow` are imported first in the same process -- a workaround already used in `train_simcse.py` and `evaluate.py`, and now also needed in the two new scripts (`seed_sts_pairs.py`, `embed_corpus.py`).
- `'make'` is not installed in this shell; every Makefile target's underlying `'python -m module --config ...'` command was run directly (documented above for each phase) and produces identical results.
- Stale Docker containers from earlier work can silently shadow freshly started local servers on the same port -- always check `'docker ps -a'` before assuming a fresh process is the one actually receiving requests.

## 4. Deliverables & how to reproduce

### Files produced

|                                          |                                             |
|------------------------------------------|---------------------------------------------|
| data/corpus_ru.txt                       | 28,283-sentence deduplicated Russian corpus |
| data/sts_test_ru.jsonl                   | 22 human-labeled STS pairs                  |
| artifacts/tokenizer_ru/                  | BPE tokenizer (vocab_size=16000)            |
| artifacts/encoder_ru/                    | pretrained MLM encoder                      |
| artifacts/simcse_ru/                     | SimCSE sentence-transformers model          |
| artifacts/embeddings_ru/embeddings.jsonl | 28,283 final embedding vectors              |
| metrics/eval_ru.json                     | Spearman / Recall@5 / MRR@5                 |

### Commits (all pushed to origin/master)

```
d781633 feat(ru): PDF extraction + ru corpus/tokenizer configs
7bcb750 feat(ru): bounded MLM pretrain
ebc528d feat(ru): unsupervised SimCSE
8727f20 feat(ru): STS labeling UI
e85d661 feat(ru): eval
3a852e7 feat(ru): embeddings deliverable + serve skew verification
```

### Quality gates -- all green

```
ruff check src tests      -> All checks passed!
ruff format --check src tests -> 42 files already formatted
mypy src                  -> Success: no issues found in 28 source files
pytest -m "not e2e"       -> 41 passed, 6 deselected
```

### Reproducing end-to-end

```
python -m langembed.data.extract_text --pdf data/raw/voina-i-mir.pdf \
  --out data/raw/voina_i_mir_ru.txt
python -m langembed.data.build_corpus --config configs/ru/tokenizer.yaml
python -m langembed.tokenizer.train_tokenizer --config configs/ru/tokenizer.yaml
python -m langembed.pretrain.train_mlm --config configs/ru/pretrain.yaml
python -m langembed.contrastive.train_simcse --config configs/ru/contrastive.yaml
docker compose up -d postgres
python scripts/seed_sts_pairs.py --config configs/ru/contrastive.yaml --n 60
# label pairs at http://localhost:8001/label, then:
curl http://localhost:8001/export-sts
python -m langembed.eval.evaluate --config configs/ru/eval.yaml
python scripts/embed_corpus.py --config configs/ru/contrastive.yaml \
  --out artifacts/embeddings_ru/embeddings.jsonl
LANGEMBED_MODEL_DIR=artifacts/simcse_ru LANGEMBED_LANG=ru \
  uvicorn langembed.serving.serve:app --port 8000
python scripts/verify_serve_skew.py --n 10
```